

Production AI Agents at Major Tech Companies & A Deep Reference Stack for Building Them

TL;DR

Part A: Every major platform vendor now runs agents in real production, not just demos — IBM (watsonx Orchestrate, with its own AskHR handling more than 11.5 million interactions in 2024 at a 94% containment rate), Google (Gemini Enterprise/ADK/A2A), Microsoft (Copilot Studio + the new Agent Framework, GA-tracking Q1 2026), AWS (Bedrock AgentCore, GA), NVIDIA (NeMo/NIM/Guardrails), Anthropic (MCP + Claude Agent SDK), and Salesforce/ServiceNow/SAP. The published engineering consensus is remarkably uniform: start simple, prefer workflows over autonomous agents, add multi-agent structure only to fix a specific failure mode, and instrument everything.

IBM's specific guidance centers on a documented Agent Development Lifecycle (ADLC), "start with single specialist agents then compose," make orchestration observable (log every decision/tool call), and layer governance (watsonx.governance) and identity as first-class concerns — codified with Anthropic in "Architecting Secure Enterprise AI Agents with MCP."

Part B: The recommended open build stack is LangChain 1.0 + LangGraph 1.0 (MIT) for orchestration, LangSmith for observability/eval, MCP (tools) + A2A (agent-to-agent) as the two complementary protocols, with Hostinger VPS viable for self-hosted single-node deployments but **not** appropriate for GPU/regulated/HA-critical workloads. OpenClaw is a genuinely important but security-troubled self-hosted agent framework that should be treated as high-risk for enterprise use.

Key Findings

Part A — What's actually shipped

IBM — watsonx Orchestrate is GA, deployable on IBM Cloud, AWS, or on-premises, connecting 80+ applications. IBM's internal AskHR handled more than 11.5 million interactions in 2024 with a 94% containment rate (only 6% routed to a human HR Partner). IBM open-sourced the BeeAI framework (from IBM Research's Bee Agent Framework) and contributes to the agent-protocol ecosystem (ACP, A2A, MCP). Granite 4.0 models (granite-4-h-small, granite-4-h-micro) are IBM's enterprise open models for agents. Project Bob ("IBM Bob") is IBM's agentic dev tool that can generate agents and MCP servers into Orchestrate.

Google Cloud — ADK is open-source (Python, TypeScript, Go, Java, Kotlin), Gemini Enterprise (formerly Agentspace) is the enterprise agent platform, Agent Engine/Agent Runtime hosts agents, and Google created A2A (April 2025) and donated it to the Linux Foundation (June 2025).

Microsoft — Copilot Studio and Azure AI Foundry Agent Service are the production surfaces; AutoGen + Semantic Kernel converged into the Microsoft Agent Framework (public preview Oct 2025, GA targeted Q1 2026), with Entra Agent ID for agent identity.

AWS — Bedrock AgentCore (Runtime, Gateway, Identity, Memory, Policy) with the Strands Agents SDK; AgentCore Policy (GA March 2026) uses Cedar for deny-by-default per-tool-call authorization. [Medium](#)

NVIDIA — NIM microservices, NeMo Agent Toolkit (formerly AgentIQ), NeMo Guardrails (used by Amdocs, Cerence, Lowe's), and AI Blueprints including the Data Flywheel Blueprint.

Anthropic — Authored "Building Effective Agents," created MCP (Nov 2024), and ships the Claude Agent SDK.

Salesforce/ServiceNow/SAP — Agentforce, ServiceNow AI Agent Orchestrator, and SAP Joule are all shipping with real customer adoption.

IBM's published engineering techniques

Single vs. multi-agent: Start simple; single agents for well-defined tasks with a narrow toolset; add multi-agent structure only where there's a clear failure mode (latency, specialization, verification, governance, coordination).

Architecture patterns: hierarchical/supervisor patterns, ReAct, ReWOO, agent control plane, agent gateway.

Observability: "Make orchestration observable" — log each decision, tool call, and intermediate result.

Lifecycle: Agent Development Lifecycle (ADLC), agent lifecycle management, AgentOps (IBM has a tutorial on AgentOps with IBM Telemetry using watsonx Orchestrate).

Governance/identity: watsonx.governance for logging, traceability, and compliance; co-authored with Anthropic "Architecting Secure Enterprise AI Agents with MCP."

Cross-industry reliability guidance

Anthropic "Building Effective Agents": workflows vs. agents distinction; five workflow patterns (prompt chaining, routing, parallelization, orchestrator-worker, evaluator-optimizer); keep it simple. [DevGenius](#)

Anthropic "Effective context engineering": curate canonical few-shot examples; keep context "informative yet tight."

12-factor agents (HumanLayer/Dex Horthy): own your prompts, own your context window, own your control flow, stateless reducer, tool calls as structured output, human-in-the-loop, small focused agents.

Details

PART A — Production agent deployments and engineering techniques

IBM

IBM's flagship enterprise agent platform is **watsonx Orchestrate**, which is generally available and can be deployed on IBM Cloud, AWS, or on-premises. IBM markets it as connecting 80+ applications and reducing manual work substantially. Its Agent Builder allows no-code agent creation, and IBM ships an Agent Development Kit (ADK) plus an agent catalog. IBM's flagship internal proof point is **AskHR**: per IBM's own case study/Think article "Embracing the future of HR by becoming an AI-first enterprise," "In 2024, AskHR handled more than 11.5 million interactions; 94% of those interactions were contained within the platform... only 6% needed to be routed outside of AskHR to a specialized HR Partner." IBM also promotes procurement, sales, and customer-care agents.

IBM's model layer is the **Granite** family; Granite 4.0 (granite-4-h-small, granite-4-h-micro) are instruction-following models built for structured/long-context and tool use, positioned as high-value small models. IBM Research open-sourced the **BeeAI framework** (evolved from the Bee Agent Framework), a framework for multi-agent orchestration with observability (Emitter) and support for the Agent Communication Protocol (ACP), which brings LangGraph, CrewAI, and BeeAI agents into one runtime and

integrates with Arize Phoenix for tracing. **Project Bob (IBM Bob)** is IBM's internal/commercial agentic developer tool that integrates into IDEs, uses Granite models for code tasks, and can generate agents and MCP servers programmatically that plug into Orchestrate. **watsonx.governance** provides the audit/logging/compliance layer.

IBM's published engineering techniques (from IBM Think, IBM Developer, and IBM's agentic architecture hub):

On architecture, IBM's "What is Agentic Architecture?" explainer (published Jan 2025, updated March 2025; authors Vanna Winland, Jess Bozorg, Cole Stryker) documents single-agent advantages (simplicity, predictability, speed, cost, fewer integration challenges) and provides a taxonomy: agent orchestration, agent control plane, hierarchical agents, multi-agent systems, ReAct, and ReWOO.

IBM's guidance is to **start with single specialist agents, evaluate their performance, then compose them into multi-agent patterns** according to a clear need.

Observability is treated as the biggest risk mitigation: log every decision, tool call, and intermediate result to debug, evaluate, and audit.

IBM documents an **Agent Development Lifecycle (ADLC)**, agent lifecycle management, agent deployment, and agent testing, plus a "build production-ready AI agent with watsonx Orchestrate" tutorial and an **AgentOps** tutorial using IBM Telemetry.

IBM co-created with Anthropic the guide **"Architecting Secure Enterprise AI Agents with MCP,"** formalizing the ADLC and multi-model orchestration (Claude for creative reasoning, Granite for domain constraints).

IBM publishes framework comparisons (CrewAI vs LangGraph vs BeeAI) on IBM Developer.

Google / Google Cloud

The **Agent Development Kit (ADK)** is open-source and available in Python, TypeScript, Go, Java, and Kotlin (Go 2.0 GA with graph workflows). **Gemini Enterprise** (previously Agentspace) is the enterprise agent platform; Google announced that going forward all Vertex AI services will be delivered through the Gemini Enterprise Agent Platform. Agent Engine / Agent Runtime hosts and manages deployed agents, and the Model Garden offers 200+ models. Google created the **Agent2Agent (A2A)** protocol (announced April 9, 2025 with 50+ partners) and donated it to the Linux Foundation on June 23, 2025, seeding the Agent2Agent project with AWS, Cisco, Google, Microsoft, Salesforce, SAP, and ServiceNow. Google publishes a Codelab showing Gemini Enterprise agents integrating with Workspace (Calendar, Gmail, Drive, NotebookLM) via ADK and a Google-managed MCP server.

NVIDIA

NVIDIA's agent stack: **NIM microservices** (optimized containerized model endpoints), **NeMo Agent Toolkit** (formerly Agent Intelligence Toolkit/AgentIQ, open-source Python), **NeMo Guardrails** (content safety, topic control, jailbreak detection — NemoGuard JailbreakDetect "is an LLM jailbreak classification model trained on a dataset of 17,000 known challenging and successful jailbreaks, built in part using NVIDIA Garak," per NVIDIA's technical blog; used by Amdocs, Cerence AI, and Lowe's), and **NeMo microservices** (Curator, Customizer, Evaluator, Retriever, Guardrails). **AI Blueprints** are reference workflows (e.g., AI Virtual Assistant, Data Flywheel Blueprint). NVIDIA's guidance emphasizes the "data flywheel" — continuously distilling, fine-tuning, and evaluating agents to move to smaller/cheaper models

over time. (NVIDIA's frequently quoted "resolves issues up to 40% faster" customer-service claim could not be located on a primary NVIDIA source and should be re-verified before use.)

Microsoft

Copilot Studio (with multi-agent orchestration and governance controls) and **Azure AI Foundry Agent Service** are Microsoft's production surfaces. **AutoGen** (research; event-driven multi-agent) and **Semantic Kernel** (enterprise orchestration) converged into the **Microsoft Agent Framework** (open source; public preview Oct 2025; both predecessors moved to maintenance mode; GA targeted end of Q1 2026, RC 1.0 landed Feb 19, 2026). It ships with OpenTelemetry observability, Entra ID authentication, long-running durability, human-in-the-loop approval, full MCP support, and five stabilized orchestration patterns. **Entra Agent ID** provides agent identity. Install via `pip install agent-framework` or `dotnet add package Microsoft.Agents.AI`. KPMG is cited as an early enterprise validator (audit automation with HITL approval).

Amazon / AWS

Amazon Bedrock AgentCore (introduced in preview, now GA components) provides Runtime (agent hosting), Gateway (connects agents to tools as MCP servers, converts APIs/Lambda to MCP tools), Identity (secure token/API-key storage), Memory, and Policy. **AgentCore Policy** (GA March 2026) uses **Cedar** (AWS's open-source authorization language, now in CNCF) for deny-by-default, per-tool-call authorization evaluated at the Gateway. **Bedrock Agents** are the fully managed higher-level option. The **Strands Agents SDK** is AWS's open-source agent framework (default in AgentCore projects), which natively supports MCP. The AgentCore CLI (`npm install -g @aws/agentcore`, Node.js 20+) can even generate Cedar policies from natural language (`agentcore add policy --generate "Only allow refunds under 1000 dollars"`).

Anthropic

Anthropic authored the influential "**Building Effective Agents**" essay, created **MCP** (Model Context Protocol, Nov 2024) — now the de facto tool-connection standard with 110M+ monthly downloads — and ships the **Claude Agent SDK** (renamed from Claude Code SDK on Sept 29, 2025; Python and TypeScript; **proprietary/Anthropic Commercial Terms**, despite some blogs describing it as open source). Anthropic's SWE-bench Verified work demonstrates agents solving real GitHub issues.

Salesforce / ServiceNow / SAP

Salesforce Agentforce — per Salesforce's Dec 3, 2025 Q3 FY26 earnings release, over 18,500 Agentforce deals closed since launch, with over 9,500 paid deals (up 50% QoQ) and 3.2 trillion tokens processed; by Q4 FY26 (Feb 25, 2026) this reached 29,000 deals closed and \$800M Agentforce ARR (up 169% Y/Y). Agentforce 3 added a command center for testing/observability and multi-agent orchestration; action-based pricing (~\$0.10/action); AgentExchange marketplace.

ServiceNow AI Agent Orchestrator + AI Agent Studio position ServiceNow as a cross-functional "control tower"; target of \$1B AI-specific revenue by 2026.

SAP Joule — 50+ Joule assistants managing 200+ specialized agents across finance, supply chain, HR, procurement, and CX; SAP uses A2A for external agent interop (though SAP has controversially routed external agents through Joule).

Cross-industry reliability engineering

Anthropic "Building Effective Agents": distinguishes *workflows* (LLMs + tools on predefined code paths) from *agents* (LLMs dynamically directing their own process); recommends the simplest solution first; documents five workflow patterns (prompt chaining, routing, parallelization, orchestrator-worker, evaluator-optimizer). Antipatterns to avoid: monolithic agents, over-engineered planning, missing observability.

Anthropic "Effective context engineering": curate diverse canonical examples over stuffing edge cases; keep context tight; dynamically retrieve context at runtime.

12-factor agents (HumanLayer, ~23.5k GitHub stars): own your prompts; own your context window (Dex Horthy's analysis of developer sessions identified a "dumb zone" in the middle 40-60% of large context windows where recall degrades — the "lost in the middle" problem); own your control flow; treat agents as stateless reducers; structured tool-call outputs; explicit human-in-the-loop; small focused single-purpose agents; make state resumable.

LangGraph production guidance: state-machine model, checkpoints for durability, interrupts for HITL, prune state aggressively (agents fail when state grows unbounded), use Annotated reducers to prevent silent overwrites. Cited production users: Uber, LinkedIn, Klarna — where, per the OpenAI/Klarna Feb 27 2024 announcement, the AI assistant handled two-thirds of Klarna's customer-service chats (2.3M conversations, equivalent to 700 full-time agents) and cut resolution time "from 11 minutes to under 2 minutes."

MCP security guidance: OAuth 2.1, per-request audience validation, exact-string redirect_uri matching, human-in-the-loop for privilege-crossing actions, short-lived scoped tokens, per-tool scopes.

PART B — Deep reference material

1. OpenClaw

What it is: A free, open-source (MIT) self-hosted autonomous AI agent framework that turns LLMs into proactive 24/7 personal assistants using messaging platforms (WhatsApp, Telegram, iMessage, Discord, Slack) as the main UI. Created by Austrian developer **Peter Steinberger** (founder of PSPDFKit/Nutrient). Name history: Warelay (Nov 24, 2025) → Clawdbot (Jan 2026; Anthropic reportedly objected to "Claude" similarity) → Moltbot (late Jan 2026) → **OpenClaw (Jan 30, 2026)**. Written in TypeScript and Swift. Repo: github.com/openclaw/openclaw; site: openclaw.ai. It became one of the fastest-growing repos in GitHub history (250,000+ stars in roughly 60 days; 295,000+ later). Steinberger joined OpenAI in February 2026 to "drive the next generation of personal agents," with OpenClaw transitioning to a foundation with OpenAI sponsorship.

Architecture: A core **AI Gateway** connects messaging apps to LLM-powered agents that take real actions. **Nodes** are remote execution hosts (typically macOS machines) paired with the gateway to run commands/control the browser. A **heartbeat daemon** drives proactive behavior (scheduled tasks, morning briefs). Behavior is defined in configuration not code — the centerpiece is a **SOUL.md** Markdown file defining the agent's identity/personality/capabilities. Model-agnostic provider configuration. Memory includes a "dreaming"/consolidation concept (converting short-term context to long-term memory during idle time). The **AgentSkill** system with the **ClawHub** skill registry allows extensibility; skills can be vibe-coded by the agent itself.

Installation: npm, Docker, one-line installers, and hosted/VPS options; web interface historically ran on port 8080 with authentication disabled by default.

Security concerns (verified, serious):

CVE-2026-25253 ("ClawJacked"): unauthenticated access to stored API tokens via the `/api/export-auth` endpoint; the WebSocket gateway defaults to exempting loopback (127.0.0.1) from strict auth, enabling malicious-link browser attacks → RCE.

Out of the box, OpenClaw bound its control interface to all network interfaces. Multiple firms found tens of thousands of exposed instances: SecurityScorecard's STRIKE found 28,663–42,900 unique IPs hosting exposed control panels across 76 countries, ~63% exploitable, 12,812–15,200 vulnerable to RCE, 549 correlated with prior breaches. Bitsight observed 30,000+ exposed instances (Jan 27–Feb 8). Hunt.io found 17,500+ instances vulnerable to CVE-2026-25253. Some reports cited 135,000–220,000+ exposed instances.

ClawHavoc: supply-chain attack flooding ClawHub with 1,184 malicious skills (~20% of the marketplace) delivering Atomic macOS Stealer.

Prompt injection: indirect prompt injection via messages/hidden web text; OpenClaw relies on frontier-model safety training and warns when small local models are used for web-touching tasks.

China's Ministry of Industry warned OpenClaw poses "significant security risks when improperly configured."

Recommended hardening: enable authentication; bind to localhost; put behind a reverse proxy with auth/TLS; sandbox non-main sessions in per-session containers with no workspace access by default and strict tool allowlists; restrict who can message the bot; rotate/scope API keys; patch to post-CVE versions.

Enterprise verdict: OpenClaw is a fascinating case study and useful for personal/experimental use, but its security track record makes it **inappropriate for enterprise production** without heavy isolation and custom hardening. Clearly separate verified facts (CVE, exposure counts, MIT license, name history) from community/marketing claims (some star counts and "dreaming" descriptions come from secondary blogs).

2. LangChain

Current version: LangChain 1.0 reached GA on **October 22, 2025** (first stable major release, no breaking changes until 2.0); the `langchain` package is on the 1.3.x line, `langchain-core` on ~1.4.x. License: **MIT**.

Architecture: `langchain-core` (base abstractions: Runnables, chat models, tools, output parsers), integration packages (e.g., `langchain-openai`), and the LCEL/Runnable interface for composing chains. The new `create_agent` is the standard agent-building abstraction in 1.0 — a simpler interface than the legacy `create_react_agent`, built on the LangGraph runtime with a middleware system (before_model/after_model/wrap_tool_call hooks, plus built-in HITL, summarization, PII-redaction middleware). Legacy chains/retrievers moved to `langchain-classic`. **RAG** is built with document loaders → text splitters → embeddings → vector stores → retrievers → chains. **LangChain vs LangGraph**: LangChain for simpler LCEL chains and standard agents; LangGraph for stateful, durable, multi-step/multi-agent orchestration requiring persistence, HITL, or parallel fan-out.

3. LangGraph

Current version: LangGraph 1.0 GA (late October 2025), MIT-licensed. It models agent workflows as a **StateGraph**: a TypedDict state schema flows through Python-function **nodes** connected by direct and **conditional edges**. **Reducers** (via `Annotated`) control how state fields merge (preventing silent overwrites in parallel branches). **Checkpointers** persist state after every node: MemorySaver (dev), SqliteSaver (single-server), PostgresSaver (multi-instance scale). **Threads** (via `thread_id`) identify sessions; the full

checkpoint history enables **time travel**. **Interrupts** (`interrupt_before`/`interrupt`) plus `update_state()` + `invoke(None, config)` implement human-in-the-loop. Supports streaming, subgraphs, and a prebuilt ReAct agent. Multi-agent patterns: supervisor, swarm, hierarchical teams. Deployment via LangGraph Platform (rebranded "LangSmith Deployment"). In 1.0, `langgraph.prebuilt` functionality moved to `langchain.agents`. Production concerns: recursion limits, cost/loop control, unbounded state growth (prune aggressively). Cited users: Uber, LinkedIn, Klarna.

4. LangSmith

What it is: LangChain's observability + evaluation platform (proprietary; GA July 2024). Handles 1B+ traces. **Tracing:** captures runs, traces, threads, and projects; instrument via `@traceable`, context managers, or RunTree API. Works beyond LangChain (OpenAI, Anthropic, Vercel AI SDK, LlamaIndex, CrewAI, Pydantic AI) and supports **OpenTelemetry** (ingest OTel/OpenLLMetry traces or export to other backends; there is a documented "evaluate with OpenTelemetry" flow). **Evaluation:** offline evals from datasets/examples, LLM-as-judge, code/human/pairwise evaluators, plus online production monitoring. **Feedback:** annotation queues route filtered traces (e.g., low-confidence LLM-judge flags) to reviewers with structured rubrics, feeding back into datasets. **Prompt management:** playground and hub. **Deployment:** managed cloud, BYOC, and self-hosted (Enterprise on Kubernetes). Pricing: free Developer tier (5,000 traces/mo), Plus \$39/user/mo. Main limitation: greatest value when LangChain-centric; can get expensive at scale (open-source Langfuse, MIT, is the common alternative — note ClickHouse acquired Langfuse, a roadmap consideration).

5. A2A (Agent2Agent protocol)

Origin: Google, announced April 9, 2025; donated to the Linux Foundation June 23, 2025 (Agent2Agent project with AWS, Cisco, Google, Microsoft, Salesforce, SAP, ServiceNow). **Spec v1.0** is the first stable production-ready release; Apache 2.0; 150+ member organizations. **Core concepts:** an **Agent Card** (structured, cryptographically signable identity + capability declaration enabling trust across org boundaries); agent discovery; **tasks** with a defined state-machine lifecycle; **messages**, **parts**, and **artifacts**; streaming via SSE; push notifications; multi-tenancy; extensions. A client agent discovers a remote agent, delegates a sub-task, and coordinates — while each agent stays opaque (no exposure of internal memory/tools/logic). **Transports:** JSON-RPC, gRPC, HTTP+JSON. **SDKs:** Python, TypeScript, Java, Go, C#/.NET. **Relation to MCP:** complementary — A2A connects agents to each other; MCP connects an agent to tools/data. Roadmap: interoperability spec, registry consolidation, security/deployment best practices. **Worked-example pattern:** Agent A fetches Agent B's Agent Card at a well-known URL → verifies capabilities/signature → sends a task (JSON-RPC `message/send`) → subscribes to SSE for streaming status updates → receives artifacts on task completion. (Note: sources conflict slightly on v1.0 timing — some cite a March 2026 release, others an April 9, 2026 one-year milestone; re-verify at build time.)

[Google Open Source](#)



6. MCP (Model Context Protocol)

Origin: Anthropic, November 2024 (public spec 2024-11-05). Uses JSON-RPC 2.0; design inspired by the Language Server Protocol. **Architecture:** hosts, clients, servers. **Three primitives:** tools (callable functions), resources (data), prompts (templates); plus sampling, roots, and elicitation. **Spec revision timeline:** 2024-11-05 (initial) → 2025-03-26 (Streamable HTTP transport replacing HTTP+SSE; OAuth 2.1 authorization) → 2025-06-18 (structured output) → **2025-11-25 (latest stable):** async Tasks primitive,

better OAuth, extensions (MCP Apps for UI), formal deprecation policy. A **2026-07-28 release** goes stateless (removes the initialize handshake and protocol-level sessions/Mcp-Session-Id, so a remote MCP server behaves like any HTTP workload and scales horizontally). **Transports:** stdio (local subprocess) and Streamable HTTP; HTTP+SSE deprecated since 2025-03-26. **Building a server:** the official SDKs (e.g., Python `FastMCP`) let you decorate functions as tools; a client connects over stdio or HTTP and lists/calls tools. **Security risk classes:** tool poisoning (malicious instructions in tool descriptions/metadata — per the arXiv study "Model Context Protocol Threat Modeling...", Hasan et al. examined 1,899 open-source MCP servers (343 official, 1,556 community) and found 7.2% contained general security vulnerabilities and 5.5% exhibited MCP-specific attack vectors such as tool poisoning), confused deputy (OAuth proxy flaw), token passthrough (forwarding tokens without audience validation). **Best practices:** OAuth 2.1, per-request audience validation (RFC 9068/8707), per-client consent, exact redirect_uri matching, HITL for privilege-crossing actions, short-lived scoped tokens. **Ecosystem:** MCP Registry; joined the Linux Foundation Agentic AI Foundation. Governance: date-based versioning, RFC/TSC process. **Relation to A2A:** complementary layers of the agentic stack.

7. Hostinger (deployment target)

VPS plans (2026): KVM-based (true resource isolation), full root access, NVMe storage. KVM 1 (~1 vCPU/4GB, intro ~\$4.99–6.49/mo, renews ~\$11.99), KVM 2 (2 vCPU/8GB, the recommended production starting point, intro ~\$8.99–12.99, renews ~\$14.99), KVM 4 (4 vCPU/16GB, renews ~\$24.99–28.99), KVM 8 (8 vCPU/32GB, renews ~\$49.99). Production stack in 2026: Ubuntu 24.04 LTS, Node.js 24 LTS. Includes hPanel, the Kodee MCP-powered AI assistant, Vanguard DDoS protection, Monarx malware scanner, free weekly backups (daily is a paid add-on), a browser web terminal, a public API, and one-click templates (MERN/MEVN pre-install PM2, Certbot, Nginx; also Docker, CyberPanel, cPanel). US data centers: Ashburn VA and Phoenix AZ. **Hostinger Horizons** is Hostinger's AI app builder (natural-language app generation) — suitable for simple web apps, not production agent runtimes. **Docker/Node/Python** all supported on VPS. **Deploying a self-hosted agent (e.g., LangGraph service or OpenClaw):** SSH in → install runtime → configure domain/DNS → SSL via Certbot/Caddy → reverse proxy (nginx/Caddy) → process management (pm2/systemd) or Docker Compose → firewall (ufw) + fail2ban → backups → monitoring. **Honest limitations:** support is unmanaged; **no GPU** availability (so no self-hosted GPU inference — you must call external model APIs); no formal enterprise-grade uptime SLA or compliance certifications comparable to hyperscalers; scaling is vertical only. **Verdict:** fine for a single-node self-hosted agent gateway or a small LangGraph API calling hosted models; **not appropriate** for GPU inference, regulated/compliance-heavy workloads, or HA-critical production — use a cloud provider (AWS/GCP/Azure) for those.

8. Other useful tools in the stack

Temporal (MIT) — durable execution platform splitting deterministic workflows from auto-retrying activities; workflows resume exactly after crashes. Highly relevant to agent reliability (checkpointing/durable execution); OpenAI uses Temporal for Codex; ships integrations with the OpenAI Agents SDK, Vercel AI SDK, and Pydantic AI (LLM calls cached on replay so you don't re-pay them). *Worth covering: it's the strongest answer to "how do my long-running COO-agent workflows survive a crash."* **Open Policy Agent (OPA)** (Apache 2.0, CNCF Graduated Jan 2021; v1.0 released) — policy-as-code engine using Rego; decouples authorization from app code. *Worth covering: the vendor-neutral analog to AWS Cedar for enterprise "COO agent" governance.*

LlamaIndex (MIT; core on 0.14.x) — data framework for RAG/agentic workflows; 300+ data connectors via LlamaHub. LlamaCloud/LlamaParse are separate commercial services. *Worth covering: best-in-class for the data/RAG layer feeding agents.*

CrewAI (MIT; independent of LangChain) — role-based multi-agent (Crews) + event-driven Flows; native MCP and A2A support. *Worth covering: a simpler multi-agent mental model than raw LangGraph.*

Pydantic AI (MIT; V1 Sept 2025, V2.0 June 23 2026, latest 2.x) — type-safe agent framework ("FastAPI for agents") with validation, DI, MCP, and native Temporal integration. *Worth covering: type safety and structured outputs matter for telemetry pipelines.*

OpenTelemetry GenAI semantic conventions (experimental/"Development," not stable; moved to a dedicated repo June 2026) — standardized spans/attributes/metrics for LLM calls, agent orchestration, and MCP tool calling. *Worth covering: the emerging vendor-neutral standard, but flag that it's not yet stable.*

Vector DBs: pgvector (open source Postgres extension; vectors alongside app data, best for a single-database COO deployment), Qdrant (Apache 2.0, Rust, best-in-class filtering), Weaviate (BSD-3, built-in hybrid search), Pinecone (proprietary managed, scales to billions).

Eval frameworks: Ragas (RAG metrics — faithfulness, context precision/recall; license reported as Apache 2.0 by most sources, MIT by one — verify against GitHub), DeepEval (Apache 2.0; pytest-style, 50+ metrics incl. agent task-completion/tool-correctness), promptfoo (MIT; prompt/agent regression testing + red-teaming).

Recommendations

For the vending-machine telemetry optimization agent (narrow, well-defined):

Start with a **single agent** (LangChain 1.0 `create_agent` or Pydantic AI for type safety) calling a small set of well-designed tools. Per IBM and Anthropic guidance, do NOT reach for multi-agent here.

Instrument from day one with **LangSmith** (or OTel + Langfuse if you want open source / data control).

If telemetry workflows are long-running or must survive crashes, wrap them in **Temporal** or use LangGraph checkpoints (PostgresSaver).

For the enterprise multi-agent "COO agent" orchestration platform:

Use **LangGraph 1.0** for orchestration (supervisor/hierarchical patterns), with PostgresSaver for durability and interrupts for human-in-the-loop approval gates on any privilege-crossing action.

Adopt **MCP** for tool/data connections and **A2A** for cross-team/cross-vendor agent interop — they're complementary and both Linux Foundation-governed.

Enforce **policy-as-code** with OPA (or Cedar if on AWS/AgentCore) for deny-by-default authorization; add agent identity (Entra Agent ID on Azure, AgentCore Identity on AWS).

Follow IBM's ADLC and "make orchestration observable" rule: log every decision/tool-call; add watsonx.governance-style audit if in the IBM ecosystem.

Build an **evaluation harness** early (Ragas for any RAG, DeepEval/promptfoo for agent behavior + red-teaming; LangSmith LLM-as-judge for online monitoring).

Deployment:

Prototype/self-host single-node services on a **Hostinger KVM 2+** VPS (Docker Compose + nginx/Caddy + systemd/pm2 + ufw/fail2ban). Move to a **hyperscaler** (AWS Bedrock AgentCore, Azure AI Foundry, or GCP Gemini Enterprise) for GPU inference, compliance, HA/SLA, and scale.

Do not run OpenClaw in enterprise production without heavy isolation; if used at all, treat as untrusted, sandbox aggressively, never expose the gateway, and patch past CVE-2026-25253.

Thresholds that change these recommendations:

Move single → multi-agent only when you hit a concrete failure mode (latency, specialization need, verification, coordination).

Move Hostinger → hyperscaler when you need GPUs, compliance certs, an uptime SLA, or horizontal scale.

Adopt Microsoft Agent Framework once it hits GA (targeted end of Q1 2026) if you're Azure/M365-centric.

Caveats

Fast-moving space: version numbers and GA/preview status change monthly. Microsoft Agent Framework GA (Q1 2026 target) and several A2A/MCP milestones should be re-verified at build time. Some version numbers (Temporal, LlamaIndex, CrewAI patch levels) came from secondary sources and should be confirmed against PyPI/GitHub.

Vendor claims vs. verified facts: productivity/revenue figures (IBM's AskHR 94% containment, Klarna's 11→2 minute resolution, Salesforce's Agentforce deal/ARR numbers) are vendor-reported; treat as directional. IBM's "reduce manual work up to 75%" is marketing language. NVIDIA's "40% faster" customer-service claim could not be confirmed on a primary source.

OpenClaw: name history, MIT license, and CVE-2026-25253 are well-documented; star counts, "dreaming," and some architecture details rely on secondary/community sources and the founder's own talks. Exposure counts vary widely by source and snapshot (17,500 to 220,000+).

Claude Agent SDK license: proprietary (Anthropic Commercial Terms), despite some blogs calling it open source.

A2A v1.0 date, OTel GenAI stability, and the **Ragas license** have conflicting/evolving reports — flagged inline.

I could not independently verify every internal-deployment metric via primary IBM/vendor filings; where only secondary sources existed, I labeled them.